

„One More Time”: Security of One-time Signature Scheme Using Run-length Encoding Under Two-message Attacks

Viktória I. Villányi

Department of Computer Algebra, Eötvös Loránd University, Budapest, Hungary
vvillanyi@inf.elte.hu

Abstract. In this paper, we examine the One-time signature scheme using run-length encoding, as proposed by Steinwandt et al., under the scenario where an adversary is allowed to obtain signatures on two messages before attempting to forge a signature on a third message. Our analysis follows the line of security discussion presented by Groot Bruinderink et al. in their paper “Oops, I Did It Again – Security of One-Time Signatures under Two-Message Attacks.” By considering various attack models and different strategies, we estimate both the attack complexity and the probability of forging a signature. Our results indicate that the signature scheme performs well under a two-message attack, making it an interesting candidate for a few-time signature scheme. Few-time signature schemes such as HORS are a fundamental building block of stateless hash-based signature schemes.

Keywords: hash-based signatures, one-time signatures, two-message attacks, post-quantum cryptography

1 Introduction

The first hash-based signature scheme was introduced by Leslie Lamport [Lam79], and several subsequent schemes were developed thereafter [Mer87, Mer89, Per01, RR02]. Initially, hash-based signature schemes were limited by the large sizes of their public keys and signatures, which rendered them less competitive compared to public-key cryptosystems such as RSA and discrete logarithm-based systems. The introduction of Merkle trees enabled to issue multiple signatures under a single public key. The concept of the Merkle tree was first proposed by Ralph C. Merkle in 1979; although the work was later published [Mer89]. However, even with this improvement, the hashed-based signature schemes remained relatively inefficient for large-scale deployment for a very long time.

Since 1994, when Peter Shor developed the quantum algorithm for factoring and solving discrete logarithm problems [Sho94], it has been known that the lifespan of cryptographic schemes whose security is based on these types of hard problems is limited. This realization has renewed interest in signature schemes

based on hash functions and has motivated the development of alternative signature schemes believed to be resistant to quantum attacks, thereby stimulating extensive research in post-quantum cryptography.

Hash-based signature schemes have undergone rapid and productive development over the past twenty years. Researchers have proposed numerous improved variants of these schemes([BGD⁺06,BDK⁺07,BDH11,HRB13]), addressing efficiency limitations and enhancing their practical applicability. Already in 2007, it was possible to cryptographically sign an unlimited number of documents (up to 2^{80}) with a Merkle-type signature scheme.

By now, several post-quantum signature schemes have already been standardized, while others are currently undergoing the standardization process. The SPHINCS+ the stateless hash-based signature was introduced in 2019 by Bernstein et al. [BHK⁺19]. In August 2024, the National Institute of Standards and Technology (NIST) finalized its standardization under Federal Information Processing Standard (FIPS) 205 [C⁺24], officially designating it as the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA).

1.1 Our contribution

Our work builds upon the research of Groot Bruinderink and Hülsing [GBH17]. In their paper, the authors investigate the security implications of reusing keys in one-time signature (OTS) schemes. While traditional security analyses assume that OTS keys are used to sign a single message, their study explores scenarios in which the same secret keys are used to sign two different messages. The authors analyze the degradation of security in prominent hash-based OTS schemes, including Lamport scheme, the optimized Lamport scheme, and the Winternitz one-time signature scheme (WOTS). They measure the security of the schemes under different security assumptions. In our paper, we examine a One-time signature scheme using run-length encoding from Steinwandt et al. [SV08] under a two-message attack, following assumptions similar to those in their study. We show that the One-time signature scheme using run-length encoding performs better than the previously examined schemes under two-message attacks. It could be a promising candidate for a few-times signature scheme to be used in stateless hash-based signature schemes under certain constraints.

2 Preliminaries

2.1 Definitions

In this section, we collect the main definitions required to construct a secure hash-based signature scheme. First, we describe the hash function along with its three main properties, and subsequently, we provide a formal definition of a signature scheme.

Definition 1 (Hash Function). *A **hash function** is a deterministic function*

$$h : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

that maps binary strings of arbitrary finite length to binary strings of fixed length n , called hash values.

The three main properties of the *cryptographic hash function*.

1. *Preimage resistance*: Given a hash value $y \in \{0, 1\}^n$, it is computationally infeasible to find any $x \in \{0, 1\}^*$ such that

$$h(x) = y.$$

2. *Second preimage resistance*: Given an input $x \in \{0, 1\}^*$, it is computationally infeasible to find a different input $x' \neq x$ such that

$$h(x) = h(x').$$

3. *Collision resistance*: It is computationally infeasible to find any two distinct inputs $x_1, x_2 \in \{0, 1\}^*$ such that

$$x_1 \neq x_2 \quad \text{and} \quad h(x_1) = h(x_2).$$

To discuss the security of a signature scheme, we first need to define the signature scheme itself.

Definition 2 (Digital Signature Scheme [KL07]). A digital signature scheme is a triple of probabilistic polynomial-time algorithms (KeyGen, Sign, Verify) satisfying the following:

1. The key-generation algorithm **KeyGen** on input of the security parameter 1^n , outputs a pair of keys a public key pk and a secret key sk , each of them has length at least n .

$$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^n)$$

2. The signing algorithm **Sign** takes as input a secret key sk and a message m from some message space $\mathcal{M}$, and outputs a signature σ .

$$\sigma \leftarrow \text{Sign}_{\text{sk}}(\text{m})$$

3. The deterministic verification algorithm **Verify** takes as input a public key pk , a message m , and a signature σ , and outputs a bit $b \in \{0, 1\}$ indicating whether the signature is valid.

$$b \leftarrow \text{Verify}_{\text{pk}}(\text{m}, \sigma)$$

Correctness: For every message $\text{m} \in \mathcal{M}$ and every key pair (pk, sk) output by $\text{KeyGen}(1^n)$ we required it holds that

$$\Pr [\text{Verify}_{\text{pk}}(\text{m}, \text{Sign}_{\text{sk}}(\text{m})) = 1] = 1.$$

Definition 3 (Security of the digital signature scheme under 2-message attack). The security of a digital signature scheme is defined by the following game between a challenger and a probabilistic polynomial time adversary $\mathcal{A}$.

1. *Setup.* The challenger runs $\text{KeyGen}(1^n)$ to obtain (pk, sk) and gives pk to $\mathcal{A}$.
2. *Query Phase.* $\mathcal{A}$ is given access to a signing oracle $\text{Sign}_{sk}(\cdot)$ and may adaptively or non-adaptively query m_1 and m_2 messages, receiving $\sigma_1 \leftarrow \text{Sign}_{sk}(m_1)$ and $\sigma_2 \leftarrow \text{Sign}_{sk}(m_2)$.
3. *Forgery.* $\mathcal{A}$ outputs a message-signature pair (m^*, σ^*) .
4. *Verify.* If $\text{Verify}_{pk}(m^*, \sigma^*) = 1$ and $m^* \notin \{m_1, m_2\}$ $\mathcal{A}$ is winning the game.

The adversary's advantage is

$$\text{Adv}_{\Sigma}^{\text{EuF-CMA}}(\mathcal{A}) = \Pr[\text{Verify}_{pk}(m^*, \sigma^*) = 1 \wedge m^* \notin \{m_1, m_2\}].$$

The scheme Σ is existentially unforgeable under chosen two-message attack (EuF-CMA) if this probability is negligible in n .

2.2 Different types of attacks

We consider the same type of attacks that were used for comparison in the paper by Groot Bruinderink and Hülising in [GBH17]. We classify the attacks into two main categories. Half of the presented attacks fall under the chosen-message attack (CMA) model. The remaining attacks are in the random-message attack (RMA) model. In the CMA setting, the adversary can choose the inputs to the signing oracle, while in the RMA setting, the inputs are chosen randomly and the signed messages are provided to the adversary. The other classification criterion relates to the goals of the adversary. The strongest attack is a full break (FB), where the adversary recovers all secret keys. In a universal forgery (UU), the adversary can forge a signature on any given message. In a selective forgery (SU), the adversary must commit to a target message before the attack begins and then forge a signature on it. In an existential forgery (EU), the adversary can forge a valid signature on some message of their choice at any time during the attack. We consider EU-CMA to represent the strongest, while FB-RMA corresponds to the weakest security notion in our model. The attacks are summarized in the Appendix A.

We can easily derive the security definitions for all these attacks with the following modifications of Definition 3:

- To derive the security definition of *existentially unforgeable under random two-message attack (EuF-RMA)* we change the **Query Phase**. $\mathcal{A}$ receives the signature σ_1 and σ_2 on two random messages, m_1 and m_2 , ($\sigma_i \leftarrow \text{Sign}_{sk}(m_i)$).
- To derive the security definition of *universal forgery under chosen message attack (UU-CMA)*, we modify the **Forgery** phase. $\mathcal{A}$ receives a random challenge message m^* and outputs a message – signature pair (m^*, σ^*) .
- To derive the definition of *selective forgery under chosen message attack (SU-CMA)* we require that the challenge message be chosen not in the forgery phase but before the **Query Phase**.

2.3 Two-message attacks

In a usual security model for digital signature schemes, the adversary is allowed to obtain signatures on multiple messages of their choice. Typically, the adversary has access to a signing oracle, enabling them to request valid signatures on arbitrary messages. The primary challenge for the adversary is then to produce a valid signature on a new message that was not previously queried. If the adversary cannot succeed in this task with more than a negligible probability, the scheme is said to be existentially unforgeable under a chosen-message attack (EuF-CMA), or simply, secure. The security of signature schemes is generally analyzed within this strongest model.

In the case of one-time signature schemes, each generated signature reveals part of the secret key. Consequently, the security is considered under a more restrictive attack model, where the adversary is permitted to query the signing oracle only once before attempting to produce a valid message-signature pair.

In our proposed model, the adversary is allowed to obtain signatures on two messages before attempting to forge a signature on a third message by using one-time signature schemes. Different strategies may be devised to obtain signatures on two correlated messages and subsequently forge a signature on a third message. From an analysis of these strategies, the attack complexity and the security of the scheme can be estimated. The adversary has access to a hash function (or hash functions). We set an upper bound on the number of accesses to the hash function and calculate the related security of the schemes. Our scheme follows the *hash-and-sign* paradigm, in which each message is first hashed and the resulting message digest is then signed. Without this initial hashing step, none of the one-time signature schemes would be secure under a two-message attack, since an adversary could easily find two related messages, derive all the secret keys through two signature queries, and then sign arbitrary messages.

3 One-time signature using run-length encoding

The One-time signature scheme using run-length encoding was proposed by Steinwandt et al. [SV08] as an alternative to the Winternitz one-time signature scheme with parameter $w = 4$, offering approximately 33% faster verification. The description of the Winternitz OTS and its predecessors can be found in Appendix B. The security of the scheme relies on the preimage and second preimage resistance of the hash function used for generating secret and public keys, as well as on the collision resistance of the hash function employed to hash the message and construct the block structure utilized.

First, we define these two hash functions for the signature scheme. The first hash function, G , is used to preprocess the message. This function maps the message space to fixed-length bit strings that appear random:

$$G : \{\text{set of possible messages}\} \rightarrow \{0, 1\}^m.$$

The second hash function, H , serves as the main building block of the one-time signature scheme. It takes secret keys as input and produces a list of derived

secret keys and, ultimately, the public key components. It is defined as

$$H : \{0, 1\}^n \longrightarrow \{0, 1\}^n,$$

The security of the scheme relies on the preimage and second preimage resistance of H and the collision resistance of G . In addition, our hash functions are assumed to behave as random oracles, producing outputs that are computationally indistinguishable from uniformly random bit strings.

To sign a message, the message must first be hashed into a predefined form. For this purpose, a random value r (counter) is concatenated with the message and then hashed together. The random value r is incremented by 1 until the first suitable hash value is obtained. To initiate the process, we define the block structure of the hashed message as

$$G(M||r) = 00||111||0||\dots||11.$$

The block representation of hashed messages ($m = G(M||r)$) is defined in the same way as in [SV08].

Definition 4 (Run-Length Representation). *Let $m \in \{0, 1\}^*$, and write m as a concatenation of non-overlapping runs of consecutive 0s or 1s, i.e.,*

$$m = B_1||B_2||\dots||B_l,$$

such that

- $B_i \in \{0\}^* \cup \{1\}^*$, and
- B_i and B_{i+1} have no substring in common, for $1 \leq i \leq l-1$.

Denoting by $|B_i|$ the length of the run B_i , we define:

$$\text{NumberOfRuns}(m) := l, \quad \text{MaxRunLength}(m) := \max\{|B_1|, |B_2|, \dots, |B_l|\}.$$

Example. If $m = (1111||000||1)$, then $B_1 = 1111$, $B_2 = 000$, $B_3 = 1$, $|B_1| = 4$, $|B_2| = 3$, $|B_3| = 1$, the number of runs 3, and the longest run has length 4. It is worth mentioning that message $(0000||111||0)$ has exactly the same structure with the same B_i and $|B_i|$ values.

The main parameters of the scheme are the output length of the hash function, denoted by m bits, and the longest expected run, which is estimated as follows:

$$\ell := \lfloor \log_2(m) - 1 \rfloor.$$

If hash output is m bit long then the scheme will be able to sign hashed message that contains exactly $R := m/2$ blocks, where each blocks are not longer than ℓ . The original version of the scheme allows the selection of an interval for the number of blocks. In [SV08], the recommended interval is $[123, 131]$, and the length of this interval (e.g., 9) is encoded in the checksum. In the present work, we restrict the interval to $[128, 128]$, and therefore omit the checksum. With this modification, the security of the scheme is enhanced when it is reused twice.

Algorithm 1 Key Generation

```
1: Input: Parameters  $R, \ell$ 
2: Output: Verification key  $pk$ , Signing key  $sk$ 
3:  $(x_1, \dots, x_R) \leftarrow \{0, 1\}^s$ 
4: for  $i = 1$  to  $R$  do
5:    $y_i \leftarrow H^{\ell-1}(x_i)$ 
6: end for
7:  $sk \leftarrow (x_1, \dots, x_R)$ 
8:  $pk \leftarrow H(y_1 \parallel \dots \parallel y_R)$ 
9: return  $(pk, sk)$ 
```

Algorithm 2 Signature Generation

```
1: Input: Message  $M$ , signing key  $sk = (x_1, \dots, x_R)$ 
2: Output: A signature  $(\sigma, r)$  for  $M$  under  $sk$ 
3:  $r \leftarrow 0$ 
4: repeat
5:    $r \leftarrow r + 1$ 
6:    $m \leftarrow G(M \parallel r)$ 
7: until  $\text{NumberOfRuns}(m) = R$  and  $\text{MaxRunLength}(m) \leq \ell$ 
8: Let  $b_i$  be the length of the  $i$ -th run in  $m$  ( $1 \leq i \leq R$ )
9:  $(\sigma_1, \dots, \sigma_R) \leftarrow (H^{\ell-b_1}(x_1), \dots, H^{\ell-b_R}(x_R))$ 
10:  $\sigma \leftarrow (\sigma_1, \dots, \sigma_R)$ 
11: return  $(\sigma, r)$ 
```

Algorithm 3 Signature Verification

```
1: Input: Message  $M$ , verification key  $pk$ , signature  $(\sigma, r)$  with  $\sigma = (\sigma_1, \dots, \sigma_R)$ 
2: Output: true if  $(\sigma, r)$  is a valid signature for  $M$ , false otherwise
3:  $m \leftarrow G(M \parallel r)$ 
4: Find  $(b_i)_{i=1}^R$  as during signing
5: if  $b_1 + \dots + b_R \neq m$  then
6:   return false
7: else
8:   for  $i = 1$  to  $R$  do
9:      $\gamma_i \leftarrow H^{b_i-1}(\sigma_i)$ 
10:   end for
11:   if  $H(\gamma_1 \parallel \dots \parallel \gamma_R) = pk$  then
12:     return true
13:   else
14:     return false
15:   end if
16: end if
```

The increased signing time We adapt the signing algorithm of the original scheme. Since the checksum has been omitted, the signing process has become more costly: the new scheme is less flexible, and signing is approximately nine times slower for a hash output length of 256 bits. However, increasing the signing cost also improves security. Without the checksum, the scheme is more resilient against certain 2-message attacks. As mentioned earlier, the restrictions that must be satisfied by the hashed message:

- The length of the longest block is only 7 bits,
- The number of the blocks is exactly 128.

The number of hash evaluations required to find an r such that $G(M||r)$ satisfies the above restrictions can be computed by Magma [BCP97]. At first, we compute the coefficient of the x^{256} term (number of hashed values satisfying the restrictions) in the polynomial

$$\left(\sum_{i=1}^7 (x^i) \right)^{128}.$$

When we divide the total number of possible bit strings of length 256 (2^{256}) by this coefficient, we obtain twice the expected number of hashes required to sign the message. Since bit strings with the same block structure but different starting bit are not distinguished in our model, the result can be halved. For example, 0111000011 is equivalent to 1000111100 in our model. Thus, the expected number of hash evaluations to find an r such that $G(m||r)$ has the correct structure is approximately is $57.23 \approx 2^{5.84}$. If an adversary is limited to τ queries to the hash function only about $\frac{\tau}{57.23}$ queries will yield useful results. We will estimate it with $\frac{\tau}{2^6}$.

Verification time We also adapt the verification algorithms. The verification time remains nearly the same; specifically, the number of hashes equals half the length of the hashed message. Compared with the Winternitz scheme with $w = 4$, this yields an improvement of approximately 36% in verification time. The main feature of this scheme was earlier that it achieves a 33% faster verification time than the Winternitz OTS. The reason of the 3% performance gain is the omission of the checksum.

Checksum As mentioned earlier, we remove the explicit checksum from our scheme. Nevertheless, the fixed structure of the hash output—consisting of exactly 128 blocks and a maximum run length of 7—serves as an implicit checksum. This implicit checksum in the One-time signature using run-length encoding provides an additional layer of security for the scheme. Without any type of checksum, an adversary could perform the following attack. Suppose the adversary obtains a valid signature on a message. From that signature, the adversary may attempt to forge a signature on another message by taking a revealed block signature value sk , computing $H(sk)$, and using the result as a part of signature

for a different message. This increases the original message length by one bit. Because we include an implicit checksum, this attack is impossible unless some other block’s run length is shortened; in that case, the adversary would have to compute $H^{j-1}(sk)$ from $H^j(sk)$, which contradicts the one-way property of the hash function H . In the single-message attack model, any adversary attempting to forge a signature must break the preimage resistance at that position.

4 Comparison between hash-based signature schemes

In this section, we compare Lamport, Optimized Lamport and Winternitz hash-based signature schemes (the description of these schemes can be found in Appendix B) with the One-time signature using run-length encoding from different perspectives.

4.1 Secret key reveal

We compare several one-time signature schemes based on the number of secret key values revealed during the signing process, assumed that the hash output length is 256 bits. The results are presented in the fourth column of Table 4.1. In the first column, the compared signature schemes are listed. The second column contains the key length expressed as the number of hash values. The bit length can be calculated by multiplying these numbers by 256 bits. The third column contains the number of hash evaluations required to generate the public key (in the case of the optimized Lamport, Winternitz, and One-time signature schemes using run-length encoding, the final hash—the hash of the public key component—is excluded).

In the fourth column, we list the verification cost, which represents the number of hash evaluations needed to verify the signature (excluding the hashing of the original message). These values correspond to the number of revealed secret keys when a signature is issued. For the Lamport, HORS [RR02], and One-time signature schemes using run-length encoding, the values are fixed and remain identical for all messages. In contrast, for the optimized Lamport and Winternitz OTS schemes, they represent estimated average values. With sufficiently high precomputation effort (e.g., by identifying an optimal target message satisfying $G(M) = 0 \dots 0$ for the signature query in case of WOTS), it becomes possible to recover nearly all secret keys with just a single signature query. In the last column, we present the ratio of the previous two quantities, which corresponds to the percentage of revealed secret keys when issuing a signature. The percentage of the secret revealed for the One-time signatures scheme using run-length encoding is calculated as follows:

$$\frac{\text{Verification cost}}{\text{Key Generation cost}} = \frac{R}{R \cdot (\log_2 R - 1)} = \frac{1}{\log_2 R - 1},$$

where R denotes the length of the public key (equal to half of the hash output size).

Signature Scheme	Key length	KeyGen cost	Verification cost	Sk revealed in %
Lamport	512	512	256	50%
Optimized Lamport	264	264	132	50%
Winternitz $w = 4$	133	399	200	50%
Winternitz $w = 8$	90	630	315	50%
Winternitz $w = 16$	68	1020	510	50%
HORS	65536	65536	16	0.02441 %
Run-length 256 $l_R = 7$	128	768	128	16.67 %
Run-length 512 $l_R = 8$	256	1792	256	14.29 %

Table 1. Comparison of signature schemes based on the number and percentage of revealed secret keys

4.2 The chain of secret keys in one-time signature schemes

We compare the structure of the secret key chains in the Winternitz OTS and the One-time signature scheme using run-length encoding.

We have the following chain of secret key in Winternitz OTS for $w=4$:

$$sk \rightarrow H(sk) \rightarrow H^2(sk) \rightarrow H^3(sk) = \text{pk component}$$

The corresponding bit strings (e.g. if the bit string in the block is '01' the signature is $H(sk)$) are signed using the keys listed above

$$00 \rightarrow 01 \rightarrow 10 \rightarrow 11,$$

and probabilities (as fractions) of revealing the secret keys in the chain are

$$\frac{1}{2^2} \rightarrow \frac{1}{2^2} \rightarrow \frac{1}{2^2} \rightarrow \frac{1}{2^2}.$$

Every secret key is used with the same probability for a signature. However, if the core secret key sk is used, then all the other values in its chain are revealed as well.

Now, we discuss the One-time signature scheme using run-length encoding ($m = 256$). The chain of secret keys:

$$sk \rightarrow H(sk) \rightarrow H^2(sk) \rightarrow H^3(sk) \rightarrow H^4(sk) \rightarrow H^5(sk) \rightarrow H^6(sk) = \text{pk comp.}$$

The corresponding bit strings are

$$0000000 \rightarrow 000000 \rightarrow 00000 \rightarrow 0000 \rightarrow 000 \rightarrow 00 \rightarrow 0,$$

or

$$1111111 \rightarrow 111111 \rightarrow 11111 \rightarrow 1111 \rightarrow 111 \rightarrow 11 \rightarrow 1.$$

The corresponding probabilities (as fractions) of revealing the secret keys in the chain are

$$\frac{1}{2^7} \rightarrow \frac{1}{2^6} \rightarrow \frac{1}{2^5} \rightarrow \frac{1}{2^4} \rightarrow \frac{1}{2^3} \rightarrow \frac{1}{2^2} \rightarrow \frac{1}{2}.$$

If block $B_i = 0$ or $B_i = 1$, we reveal the public key component $H^6(sk)$. The probability of this event is 50%. If block $B_i = 1111111$ or $B_i = 0000000$, the core secret key sk will be revealed, but the probability of this event is approximately 0.78%. If this happens, all secret key values in the chain become exposed. By comparison, in the Winternitz OTS, there is a 25% (i.e., $1/w$) chance that all secret key values in the chain is exposed, when the core secret key sk is revealed. Our intuition about security is that the signature scheme does not require all underlying problems to be hard (preimage and second preimage resistance); it remains secure provided that at least one hard problem exists.

4.3 Comparison of security of the hashed-based signature schemes

Our comparison table is based on the comparison tables presented in [GBH17], which, with minimal changes, are presented in Appendix C. In our comparison table, we present concrete values of the attack complexity (the number of hash function evaluations) and the probability of successful attacks at the same time. If the exponent of an expression is positive, then the given value corresponds to the attack complexity (the associated success probability is at least $1/2$); if it is negative, it represents the success probability. The concrete values assume a hash function with an output length of 256 bits in all cases and $w = 4$ for the Winternitz OTS. All exponents are rounded up.

Security Goal	Lamport	Optimized Lamport	Winternitz OTS	Run-length
EU-CMA	2^{36}	2^{17}	2^{16}	2^{23}
SU-CMA	2^{36}	2^{17}	2^{16}	2^{23}
FB-CMA	2^{128}	2^{55}	2^{79}	impossible
EU-RMA	2^{106}	2^{51}	2^{47}	2^{56}
UU-CMA	2^{128}	2^{55}	2^{79}	2^{83} and 2^{-56} ¹
SU-RMA	2^{-106}	2^{264}	2^{264}	2^{255} and 2^{-53} ²
UU-RMA	2^{-106}	2^{-51}	2^{-47}	2^{-56}
FB-RMA	2^{-256}	2^{-110}	2^{-158}	impossible

Table 2. Concrete values.

5 Security of the One-time signature using run-length encoding under different attacks

In this section, we examine the security of the One-time signature scheme using run-length encoding from Steinwandt et al. [SV08] under different types of two-messages attack that were listed in Subsection 2.2.

¹ Both m and r were given to the adversary.

² Message M was used as an optimal challenge message, where $H(M) = 00||11||00||...||11$ or $H(M) = 11||00||11||...||00$.

5.1 Full break under chosen message attack (FB-CMA)

First, we consider the full break of the scheme under a chosen-message attack (FB-CMA). If the hash output length is 256 bits then the longest run is $\ell = 7$. In our model, we aim to identify the two optimal messages for the signing oracle queries. The hash of these messages must also satisfy the following constraints:

1. Lengths of the hashed messages are 256 bits.
2. Each 256-bit hash value is divided into 128 blocks (runs).
3. The length of the blocks of the hashed messages (in bits) must fall into the [1 to 7] interval.

To achieve a full break, the adversary must obtain the core secret keys (sk) of all secret key chains introduced in Subsection 4.2, which requires that every block in at least one of the messages has length 7. We demonstrate that this condition cannot be satisfied when the adversary is restricted to only two message queries.

Let x_i denote the number of blocks of length i . Our goal is to maximize x_7 and satisfy the following three constraints:

$$\begin{aligned} x_1, x_2, \dots, x_7 &\geq 0 \\ \sum_{i=1}^7 i \cdot x_i &= 256 \\ \sum_{i=1}^7 x_i &= 128 \\ (x_7) &\rightarrow \max \end{aligned}$$

We solved the optimization problem using the simplex algorithm and obtained the following solution:

$$x_1 = 106, x_3 = 1, x_7 = 21, x_2 = x_4 = x_5 = x_6 = 0$$

This means that the maximum number of 7-bit blocks in a single message is upper-bounded by 21, which implies that it is impossible to perform a full break with two messages, since the maximal possible number of blocks of length 7 is only $2 \cdot 21 = 42$, instead of the required 128 (all blocks). In order to recover the secret key and achieve a full break at least $\lceil 128/21 \rceil = 7$ signatures on such special messages are needed. The hash values of all the 7 messages should contain 21 blocks of length 7. These blocks should be non-overlapping so in total they cover the 128 blocks. The number of these special messages is

$$\frac{2 \cdot 128!}{106! \cdot 21! \cdot 1!} \approx 2^{87}.$$

The probability that a single message hash has 21 length-7 blocks, 106 length-1 blocks, and 1 length-3 block is

$$\frac{2^{87}}{2^{256}} = 2^{-169}.$$

The expected computational cost of obtaining seven messages of this type (disregarding the non-overlapping restriction) is $7 \cdot 2^{169}$ hash evaluations. Therefore, the full break remains impossible, even under a seven-message attack.

5.2 Full break under random message attack (FB-RMA)

The impossibility of the full break under chosen message attack implies the impossibility of the full break under the random message attack.

5.3 Existential forgery under chosen message attack (EU-CMA)

It was shown in [SV08] that, when obtaining a signature on one chosen message it remains computationally impossible to forge a signature on a different message. Now, we assume that the adversary can obtain signatures on two chosen messages, and its challenge is to forge a signature on a third message. If the adversary can choose the query messages and the challenge message such that, at a given position the block in the query messages is longer than the block in the challenge message, then the adversary can easily forge the signature for that position. In our model, the $| (B_1)_i | < | (B_2)_i |$ means the i -th block of the $G(M_1 || r_1)$ hashed message is shorter than the i -th block of the $G(M_2 || r_2)$, but for better readability, we omit the $| \cdot |$ notation. By $(B_1)_i$, we mean (in the rest of subsection) the length of the block, without explicitly indicating it using the $| \cdot |$ symbols. We measure the probability of forging a signature on a challenge message, M_1 , when the adversary obtained signatures on two messages, M_2 and M_3 . First, the adversary hashes all three messages, $G(M_1 || r_1)$, $G(M_2 || r_2)$ and $G(M_3 || r_3)$ and constructs the block structure of these messages.

$$\begin{aligned}
& \Pr [(B_1)_i \leq (B_2)_i \vee (B_1)_i \leq (B_3)_i] \\
&= \sum_{j=1}^{\lfloor \log_2(m) - 1 \rfloor} \Pr [(B_1)_i \leq (B_2)_i \vee (B_1)_i \leq (B_3)_i \mid (B_1)_i = j] \Pr [(B_1)_i = j] \\
&= \sum_{j=1}^{\lfloor \log_2(m) - 1 \rfloor} (1 - \Pr [(B_1)_i > (B_2)_i \wedge (B_1)_i > (B_3)_i \mid (B_1)_i = j]) \Pr [(B_1)_i = j] \\
&= \sum_{j=1}^{\lfloor \log_2(m) - 1 \rfloor} \frac{1}{2^j} \left(1 - \left(1 - \frac{1}{2^{j-1}} \right)^2 \right) = 4 \sum_{j=1}^{\lfloor \log_2(m) - 1 \rfloor} \left(\frac{1}{4^j} - \frac{1}{8^j} \right)
\end{aligned}$$

If the hash output length is m -bit and the scheme is defined over $m/2$ blocks, then the probability of breaking the signature scheme in the two-message attack model is

$$\Pr [\text{break}(M_1, M_2, M_3) = 1] \approx 2^m \left(\sum_{j=1}^{\lfloor \log_2(m) - 1 \rfloor} \left(\frac{1}{4^j} - \frac{1}{8^j} \right) \right)^{m/2}.$$

For a hash function with output length of 256 bits and a message of 128 blocks, the overall probability of a forgery is

$$2^{256} \left(\sum_{j=1}^7 \left(\frac{1}{4^j} - \frac{1}{8^j} \right) \right)^{128} \approx 0.7619^{128} \approx 2^{-50}.$$

Since, in the signing procedure of the One-time signature scheme using run-length encoding, the expected number of hash evaluation needed to find r such that hash value $G(M||r)$ has the right form is $57.23 \approx 2^6$ and an adversary is limited to issue τ queries to the hash function will obtain only about $\frac{\tau}{2^6}$ useful results. Following the approach of the paper [GBH17], we perform the following calculation. The adversary choose three messages. All three messages will be hashed and two of them will be queried to the signing oracle, and the adversary will attempt to forge the signature on the third element. There are $\binom{\frac{\tau}{2^6}}{2}$ possible choices for the first two messages, and $\frac{\tau}{2^6} - 2$ choice for the challenge message. Altogether, the probability of forging a signature is

$$\binom{\frac{\tau}{2^6}}{2} \cdot \left(\frac{\tau}{2^6} - 2 \right) \cdot 2^{-50}.$$

We bound this probability by $\frac{1}{2}$ (as the adversary's success probability) and compute an upper bound for τ

$$\binom{\frac{\tau}{2^6}}{2} \cdot \left(\frac{\tau}{2^6} - 2 \right) \cdot 2^{-50} < \frac{1}{2}.$$

After simplification, we derived an estimation $\tau < 2^{22.66}$.

5.4 Selective forgery under chosen message attack (SU-CMA)

The knowledge of the public key provides no advantage to the adversary, and consequently, the complexity of selective forgery is equivalent to that of existential forgery under a chosen-message attack.

5.5 Universal forgery under random message attack (UU-RMA)

Here, all three messages are chosen randomly and given to the adversary along with signatures on two of them. The challenge of an adversary is to forge the signature on the third message. The probability of a successful universal forgery under a random-message attack (UU-RMA) equals the probability that three independently chosen random messages satisfy the break condition, which is approximately

$$\Pr [\text{break}(M_1, M_2, M_3) = 1] \approx 2^m \left(\sum_{i=1}^{\lfloor \log_2(m) - 1 \rfloor} \left(\frac{1}{4^i} - \frac{1}{8^i} \right) \right)^{m/2}.$$

This case is similar to the earlier EU-CMA case, but here the adversary has no control over the choice of the signed messages. The forging probability is 2^{-56} . However, if the adversary is able choose a value r to derive $G(M||r)$, they are able to influence the structure of the hashed message. In this case, the attack model is similar to the EU-RMA model, where we measure the computational effort required instead of the success probability.

5.6 Existential forgery under random message attack (EU-RMA)

As mentioned, this setting is very similar to the previous (UU-RMA) setting. Here, we also analyze the probability of a break involving three messages, but in this case the adversary is allowed to choose the challenge message. We are interested in the computational effort required to find a challenge message, which corresponds to the reciprocal of the break probability:

$$\frac{1}{\Pr [\text{break}(M_1, M_2, M_3) = 1]} \approx 2^{-m} \left(\sum_{i=1}^{\lfloor \log_2(m) - 1 \rfloor} \left(\frac{1}{4^i} - \frac{1}{8^i} \right) \right)^{-m/2}.$$

For the concrete parameter choice $m = 256$, this evaluates to approximately 2^{50} . After adjusting it with the effort of obtaining r such that $G(M||r)$ is a properly formed hashed challenge message (an additional filtering factor), the final number of hash queries, the complexity of the attack, is about 2^{56} .

5.7 Selective forgery under random message attack (SU-RMA)

In this model, an adversary is allowed to choose the challenge message for the forgery, before receives signatures on two random messages. The best strategy to choose a challenge message whose signature can be most easily forged using the later obtained random secret keys. In the case of the Winternitz OTS scheme, this optimal message is the one whose hash value is the all-one bit string. The analog in our setting would be when all 128 block length values are equal to 1, i.e. an alternating pattern of 1-bit blocks across the 128 blocks. However, this is impossible because the hash output (256 bits) is twice the size of this bitstring. Our best guess for the challenge message is one whose hash consists of 2-bit blocks, i.e., an alternating pattern of 2-bit blocks across the 128 blocks (totaling 256 bits). We therefore look for M and r such that

$$G(M||r) := 00||11||00||11... \text{ or } 11||00||11||00...,$$

whichever occurs first. The success probability of finding one of these messages is

$$\frac{2}{2^{256}} = 2^{-255}.$$

The expected computational effort is therefore about 2^{255} hash evaluations. Even if this challenge message is found, a 100% success probability for forging a signature on a third message cannot be achieved.

Let M_1 and M_2 be the two random messages for which the adversary receives signatures. The block structure of these hashed messages:

$$G(M_1||r_1) = (B_1)_1||(B_1)_2||\dots|(B_1)_{128} \text{ and } G(M_2||r_2) = (B_2)_1||(B_2)_2||\dots|(B_2)_{128}$$

We create a cover message, M_c from M_1 and M_2 , which is not be a real message, it is more like a structure.

Let

$$|(B_c)_i| := \max\{|(B_1)_i|, |(B_2)_i|\} \text{ for all } 1 \leq i \leq 128$$

and

$$M_c \leftrightarrow (B_c)_1||(B_c)_2||\dots|(B_c)_{128},$$

the $|(B_j)_i|$ means the length of the block i in message j . If $|(B_c)_i| = 1$ appears in the list then M_c is not a perfect cover for our chosen messages M . In general we can say M_c is a cover of M , if for all $i \in \{1, \dots, 128\}$ $B_i \leq (B_c)_i$, where $G(M||r) = B_1||B_2||\dots|B_{128}$. The success probability will be computed by the probability of the multiple appearance of $|(B_c)_i| = 1$. First, we compute the following probabilities for the cover message

$$\Pr((B_c)_i = 1) = \frac{1}{4} \text{ and } \Pr((B_c)_i \neq 1) = \frac{3}{4}.$$

We need the cover message that does not contain blocks of length 1. The probability of this event is

$$\Pr((B_c)_i \neq 1)^{128} = \left(\frac{3}{4}\right)^{128} \approx 2^{-53}.$$

If the adversary chooses a different challenge message that contains a block of length 1, then it must also contain a block of length 3, due to the structural constraint in our assumption. The rough estimation of the probability of creating a perfect cover message when the challenge message contains a 3 bit long block

$$\Pr((B_c)_i \neq 1)^{126} \cdot \Pr((B_c)_i = 1 \vee (B_c)_i \neq 1) \cdot \Pr((B_c)_i > 2) = \left(\frac{3}{4}\right)^{126} \cdot \frac{7}{16},$$

where

$$\Pr((B_c)_i > 2) = 1 - \left(\frac{1}{2} + \frac{1}{4}\right)^2 = 1 - \left(\frac{3}{4}\right)^2 = \frac{7}{16}.$$

Let us compare the probabilities of creating a cover message when the challenge message contains only 2 bits blocks and when it contains a single block of length 1:

$$\left(\frac{3}{4}\right)^{126} \cdot \left(\frac{3}{4}\right)^2 \text{ versus } \left(\frac{3}{4}\right)^{126} \cdot \frac{7}{16}.$$

The success probability decreases, meaning that forgery becomes harder when the chosen challenge message contains blocks other than 2-bit ones. Indeed,

$$\left(\frac{3}{4}\right)^2 > \frac{7}{16}.$$

Summary of results: Finding an optimal challenge message requires approximately 2^{255} hash evaluations. The success probability of the forgery is about 2^{-53} .

5.8 Universal forgery under chosen message attack (UU-CMA)

In this attack model, the adversary has the flexibility to select two optimal messages for signature queries. Their challenge is to forge a signature on any given message. The adversary's goal is to obtain secret keys via signature queries that cover the largest portion of the key space. Identifying these optimal messages requires extensive precomputation. The computational complexity of the attack is evaluated in terms of this precomputation cost. Furthermore, the effectiveness of the attack is quantified by its probability of success, which depends on the fraction of the key space that remains uncovered. We aim to minimize and precomputation cost and while maximizing the attack success probability simultaneously.

The adversary retains a certain degree of freedom. Even when receiving a random message, the adversary can still choose the value of r in the signature generation process to obtain a hash value $G(M||r)$ with a structure that is more favorable for forgery. Consequently, in this case, the model collapses into the EU-CMA model, if no restrictions are imposed on how or when r is selected. This special case is not considered further. Stronger security models can be defined by imposing stricter assumptions. For instance, the adversary may be required to complete all signing queries before receiving the challenge message. An even stricter model assumes that both the message M and the randomness r are provided to the adversary after the signature queries; in this setting, the entire structure of $G(M||r)$ is determined independently of the adversary's choices. When the adversary is permitted to choose a preferable value of r , we compute the precomputation cost. In the other case, when r is also provided, we compute the success probability. These two values are reciprocals of each other when the challenger is honest. An honest challenger samples r uniformly at random and does not attempt to select an r that makes forgery exceptionally difficult.

At first our aim is to determine the optimal cover message, along with the corresponding pair of messages to be used for the signature queries. Our guess is that the optimal cover message is the one that covers the most frequently utilized positions within the secret key chains introduced in Subsection 4.2. The relevant parameters are summarized in Table 3. The first column lists the possible block lengths $i \in \{1, \dots, 7\}$, the second column provides the corresponding probabilities $\frac{1}{2^i}$ of a block of length i , and the third column indicates the expected number of occurrences $\frac{128}{2^i}$ of such blocks, assuming the scheme based on 128 blocks. Our heuristic selects a cover message that covers all blocks of length at most 3. On average, such a cover message will cover $64 + 32 + 16 = 112$ blocks of a random message, leaving approximately $8 + 4 + 2 + 1 = 15$ blocks of length 4 or greater uncovered. These 15 longer blocks collectively account for

$$4 \cdot 8 + 5 \cdot 4 + 6 \cdot 2 + 7 \cdot 1 = 71$$

Table 3. Probability distribution and corresponding numbers for block lengths.

Length	Probability	Expected Number
1	0.5	64
2	0.25	32
3	0.125	16
4	0.0625	8
5	0.03125	4
6	0.015625	2
7	0.0078125	1

bits of the message.

The precomputation cost of finding a cover message. We attempt to find two messages from which we can derive a cover message composed of at least 3 bits long blocks. We analyze the probability that the hash of two messages can cover all 3 bits long blocks of a cover message. The probability that a bitstring contains a block of length at least 3 in either of the two messages is

$$\Pr[|(B_i)_1| \geq 3 \vee |(B_i)_2| \geq 3] = 1 - \left(\frac{3}{4}\right)^2 = \frac{7}{16}.$$

The cover messages contains 128 blocks which makes the final probability

$$\left(\frac{7}{16}\right)^{128} \approx 2^{-153}.$$

Since the structure of the messages is forced, the actual probability could be even smaller.

With the same idea as before, we can estimate the number of hash queries, denoted by τ . The success probability is bounded by $1/2$, and the adversary has a freedom to choose both message to be signed.

$$\left(\frac{\frac{\tau}{2^6}}{2}\right) \cdot 2^{-153} \leq \frac{1}{2}$$

We solve for τ :

$$\frac{\tau}{2^6} \leq \frac{1}{\sqrt{2^{-153}}} \Rightarrow \tau \leq 2^{82.5}$$

Cost of finding the optimal r value for a given challenge message. In the less strict model, the adversary can search for a random value r such that the hash value $G(M||r)$ has a structure that is easier to forge, in particular one that contains no blocks longer 3 bits. Table 4 lists, in the first column, all possible values for the longest run ℓ , and in the second column, the probabilities that the length of the longest run in $G(M||r)$ is at most ℓ . An adversary seeking a hash whose longest run is at most ℓ therefore expects to find a suitable r after about $1/\Pr(B_i \leq \ell)$ independent trials on average. If the adversary invest 2^{56} hash

Table 4. Probability of the longest run is at most l

Longest run ℓ	$\forall 1 \leq i \leq 128 \Pr(B_i \leq \ell)$
7	2^{-6}
6	2^{-8}
5	2^{-12}
4	2^{-23}
3	2^{-56}
2	2^{-255}
1	impossible

computation then using the earlier obtained signatures able to forge a signature with 100% success probability when provided the cover message contains no runs shorter than 3. The values presented in Table 4 were computed using the Magma computer algebra system [BCP97].

Summary of results: Finding an optimal cover message for all cases requires approximately 2^{83} hash evaluations. The success probability of the forgery in the strictest model is 2^{-56} . In the less strict model, the success probability of the forgery could reach 100%, but an additional 2^{56} hash computations are required.

6 Conclusion

We analyzed the One-time signature scheme using run-length encoding and compared it with the most prominent hash-based one-time signature schemes under the two-message attack. The security of the scheme was evaluated across different types of attack. The results were very encouraging: for example, the scheme fully resists full-break attacks under the chosen two-message attack. It also performed well in comparisons with the Winternitz OTS under multiple types of attacks, when evaluated in terms of attack complexity (measured by the number of hash evaluations) and the success probability of forgery. Among the different attack models, the EU-RMA model most closely captures realistic attacks against the One-time signature scheme using run-length encoding, given its structural properties, in particular the signer’s choice of a random value during the signing process. With a simple calculation, it can also be shown that even under a six-message attack in the EU-RMA model, using SHA-256 as the hash function H and a hash function G with output length 1536 bits (the signature size is $768 \cdot 256$ bits = 24.576 KB), the signature forging probability is 2^{-127} .

Acknowledgements

The author is grateful for the support of the National Research, Development and Innovation Office within the framework of the Thematic Excellence Program 2021 - National Research Subprogramme: “Artificial intelligence, large networks, data security: mathematical foundation and applications”.

References

- BCP97. Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993).
- BDH11. Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. Xmss—a practical forward secure signature scheme based on minimal security assumptions. In *International Workshop on Post-Quantum Cryptography*, pages 117–129. Springer, 2011.
- BDK⁺07. Johannes Buchmann, Erik Dahmen, Elena Klintsevich, Katsuyuki Okeya, and Camille Vuillaume. Merkle signatures with virtually unlimited signature capacity. In *International Conference on Applied Cryptography and Network Security*, pages 31–45. Springer, 2007.
- BGD⁺06. Johannes Buchmann, Luis Carlos Coronado García, Erik Dahmen, Martin Döring, and Elena Klintsevich. Cmss—an improved merkle signature scheme. In *International Conference on Cryptology in India*, pages 349–363. Springer, 2006.
- BHK⁺19. Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- C⁺24. David Cooper et al. Stateless hash-based digital signature standard. 2024.
- GBH17. Leon Groot Bruinderink and Andreas Hülsing. “oops, i did it again”—security of one-time signatures under two-message attacks. In *International Conference on Selected Areas in Cryptography*, pages 299–322. Springer, 2017.
- HRB13. Andreas Hülsing, Lea Rausch, and Johannes Buchmann. Optimal parameters for xmss mt. In *International conference on availability, reliability, and security*, pages 194–208. Springer, 2013.
- KL07. Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography: principles and protocols*. Chapman and hall/CRC, 2007.
- Lam79. Leslie Lamport. Constructing digital signatures from a one way function. 1979.
- Mer87. Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.
- Mer89. Ralph C Merkle. A certified digital signature. In *Conference on the Theory and Application of Cryptology*, pages 218–238. Springer, 1989.
- Per01. Adrian Perrig. The biba one-time signature and broadcast authentication protocol. In *Proceedings of the 8th ACM Conference on Computer and Communications Security*, pages 28–37, 2001.
- RR02. Leonid Reyzin and Natan Reyzin. Better than biba: Short one-time signatures with fast signing and verifying. In *Australasian Conference on Information Security and Privacy*, pages 144–153. Springer, 2002.
- Sho94. Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- SV08. Rainer Steinwandt and Viktória I Villányi. A one-time signature using run-length encoding. *Information Processing Letters*, 108(4):179–185, 2008.

A Classification of attacks

Here, we summarize the different types of attacks in one table.

Attack Type	CMA (Chosen-Message Attack)	RMA (Random-Message Attack)
Full Break (FB)	Adversary recovers all secret keys after chosen queries to the signing oracle.	Adversary recovers all secret keys from randomly provided signed messages.
Universal Forgery (UU)	Adversary can forge a valid signature on any given message after chosen queries.	Adversary can forge a valid signature on any given message after observing random signatures.
Selective Forgery (SU)	Adversary commits to a message before the attack and then forges a signature on this message using chosen queries.	Adversary commits to a message before the attack and forges a signature on this message after observing random signatures.
Existential Forgery (EU)	Adversary can forge a valid signature on some message of their choice at any time using chosen queries.	Adversary can forge a valid signature on some message of their choice after observing random signatures.

Table 5. Classification of attacks.

B Hash-based signature schemes

B.1 Lamport One-Time Signature Scheme

The Lamport One-Time Signature scheme was the first hash-based digital signature scheme. It provides post-quantum security under the assumption that the underlying hash function is preimage-resistant. The scheme consists of three main algorithms: key generation, signing, and verification.

Key Generation. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a cryptographic hash function and let the message length be m bits. The signer generates $2m$ random n -bit secret values:

$$\text{SK} = \{sk_{i,0}, sk_{i,1} \mid i = 1, \dots, m\},$$

where each $sk_{i,b} \in \{0, 1\}^n$ is chosen uniformly at random. The corresponding public key consists of the hashes of these values:

$$\text{PK} = \{pk_{i,0}, pk_{i,1} \mid i = 1, \dots, m\}, \quad \text{where } pk_{i,b} = H(sk_{i,b}).$$

Signing. To sign a message $M \in \{0, 1\}^m$, the signer computes

$$\sigma = \{sk_{i, M_i} \mid i = 1, \dots, m\},$$

for each bit M_i of the message, the signer reveals the corresponding secret value.

Verification. Given a message M and a signature σ , the verifier checks that:

$$H(\sigma_i) = pk_{i, M_i} \quad \text{for all } i = 1, \dots, m.$$

If all checks hold, the signature is accepted.

Security and Limitations. The Lamport signature scheme is secure against existential forgery under a one-time use assumption, provided H is preimage-resistant. The scheme's main drawbacks are the large key ($2mn$ bits) and signature sizes (mn bits).

B.2 Optimized Lamport Signature Scheme

The optimized Lamport signature scheme is similar to the original Lamport signature scheme. However, in this variant, only one secret value is used for each bit of the message instead of two. To prevent forgery, a checksum is appended to the message, and this checksum is signed in the same way as the message.

Key Generation. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a cryptographic hash function and let the message length be m bits. The signer generates ℓ random n -bit secret values:

$$SK = \{sk_i \mid i = 1, \dots, \ell\},$$

where each $sk_i \in \{0, 1\}^n$ is chosen uniformly at random, and $\ell = m + \lceil \log m \rceil + 1$. The corresponding public key consists of the hashes of these values:

$$PK = \{pk_i \mid i = 1, \dots, \ell\}, \quad \text{where } pk_i = H(sk_i).$$

Signing. To sign a message $M \in \{0, 1\}^m$, the signer computes the C checksum, which corresponds to the number of zeros in the message, and appends it to the M . Let M_i to be the i -th bit of the message.

$$C = \sum_{i=1}^m \neg M_i, \quad M^* = M \| C.$$

The signature consists of the secret key element if the corresponding bit in M^* is 1, and the public key element otherwise:

$$\sigma = (\sigma_1, \dots, \sigma_m) \quad \text{with} \quad \sigma_i = \begin{cases} sk_i, & \text{if } M_i = 1, \\ pk_i, & \text{if } M_i = 0. \end{cases}$$

Verification. To verify a signature, the verifier recomputes the checksum and verifies whether the full public key is obtained by hashing the elements of the signature that correspond to 0 bits in M^* :

$$pk_i = \begin{cases} H(\sigma_i), & \text{if } M_i = 1, \\ \sigma_i, & \text{if } M_i = 0. \end{cases}$$

If all checks hold, the signature is accepted.

B.3 Winternitz One-Time Signature Scheme

The Winternitz one-time signature (OTS) scheme is a generalized version of the optimized Lamport OTS scheme. It reduces the signature size at the cost of increased verification time, due to a higher number of hash function evaluations. The signature size optimization is achieved by representing each message using blocks of length $\log_2(w)$ instead of individual bits ($w = 2$), and by adding a small checksum to detect and prevent certain forgery attempts. Winternitz OTS with parameter $w = 2$ is equivalent to the optimized Lamport OTS.

Parameters. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a cryptographic hash function. Let B_i denote the i -th block of the message of length m bits, where each block has a value represented by $\log_2(w)$ bits.

Key Generation. The signer generates ℓ random n -bit secret values:

$$SK = \{sk_i \mid i = 1, \dots, \ell\}, \quad sk_i \in \{0, 1\}^n,$$

where

$$\ell = \ell_1 + \ell_2, \quad \ell_1 = \left\lceil \frac{m}{\log(w)} \right\rceil, \quad \ell_2 = \left\lceil \frac{\log(\ell_1(w-1))}{\log(w)} \right\rceil + 1.$$

Let ℓ_1 denote the number of message blocks, and let ℓ_2 denote the number of checksum blocks. For each sk_i , the signer computes a chain of $w - 1$ hashes:

$$pk_i = H^{w-1}(sk_i),$$

where $H^k(x)$ denotes applying H iteratively k times and $H^0(sk_i) = sk_i$. The public key is defined as

$$PK = \{pk_i \mid i = 1, \dots, \ell\}.$$

Checksum. To guarantee security against forgery, a checksum is appended to the message before signing. The checksum value is computed as

$$C = \sum_{i=1}^{\ell_1} (w - 1 - B_i).$$

The message is represented by B_i values, $M = (B_1 || B_2 || \dots || B_{\ell_1})$ with $B_i \in \{0, 1, \dots, w - 1\}$. The concatenation $(M || C)$ is then signed as a single message of length ℓ .

Signing. To sign the message M , the signer first produces the blocks of the concatenated value $(M||C) = (B_1||B_2||\dots||B_\ell)$, as mentioned earlier. Then, the signer computes the following signature components:

$$\sigma_i = H^{B_i}(sk_i), \quad i = 1, \dots, \ell.$$

The signature is defined as

$$\sigma = \{\sigma_i \mid i = 1, \dots, \ell\}.$$

Verification. Given (M, σ) , the verifier computes

$$H^{w-1-B_i}(\sigma_i)$$

for all $i = 1, \dots, \ell$, and checks that the resulting set equals the public key PK.

Security and Properties. The Winternitz OTS achieves the same one-time security level as the original Lamport OTS, while reducing the public key and signature sizes. Security relies on the preimage and second preimage resistance of the hash function H and the integrity of the checksum mechanism, which ensures that an adversary cannot modify a message without detection.

C Comparison Tables

Here, we present the comparison tables, with minimal changes, as presented in Groot Bruinderink and Hülsing's paper [GBH17] regarding the Lamport, optimized Lamport, and Winternitz one-time signature schemes, with the same focus on attack complexity and the probability of successful attacks.

The optimized Lamport Signature Scheme can be regarded as a special case of the Winternitz OTS with parameter $w = 2$; this is evident even from the tables. The first table shows the attack complexity measured in the number of hash queries.

Security Goal	Lamport	Optimized Lamport	Winternitz OTS
EU-CMA	$O\left(\left(\frac{3}{4}\right)^{-\frac{m}{3}}\right)$	$O\left((7/8)^{-\frac{m+\log m}{3}}\right)$	$O\left(\frac{(w+1)(4w-1)}{6w^2}^{-\frac{m+\log m}{3\log w}}\right)$
SU-CMA	$O\left(\left(\frac{3}{4}\right)^{-\frac{m}{3}}\right)$	$O\left((7/8)^{-\frac{m+\log m}{3}}\right)$	$O\left(\frac{(w+1)(4w-1)}{6w^2}^{-\frac{m+\log m}{3\log w}}\right)$
UU-CMA	$O\left((1/2)^{-m/2}\right)$	$O\left((3/4)^{-\frac{m+\log m}{2}}\right)$	$O\left(\left(1 - \left(\frac{w-1}{w}\right)^2\right)^{-\frac{m+\log m}{2\log w}}\right)$
FB-CMA	$O\left((1/2)^{-m/2}\right)$	$O\left((3/4)^{-\frac{m+\log m}{2}}\right)$	$O\left(\left(1 - \left(\frac{w-1}{w}\right)^2\right)^{-\frac{m+\log m}{2\log w}}\right)$
EU-RMA	$O\left(\left(\frac{3}{4}\right)^{-m}\right)$	$O\left((7/8)^{-(m+\log m)}\right)$	$O\left(\frac{(w+1)(4w-1)}{6w^2}^{-\frac{m+\log m}{\log w}}\right)$
SU-RMA	-	$O\left((1/2)^{-(m+\log m)}\right)$	$O\left(\left(\frac{1}{w}\right)^{-\frac{m+\log m}{\log w}}\right)$
UU-RMA	-	-	-
FB-RMA	-	-	-

Table 6. Attacks complexity.

The next table presents the success probabilities of various attacks on the different schemes.

Security Goal	Lamport	Optimized Lamport	Winternitz OTS
EU-CMA	1/2	1/2	1/2
SU-CMA	1/2	1/2	1/2
UU-CMA	1/2	1/2	1/2
FB-CMA	1/2	1/2	1/2
EU-RMA	1/2	1/2	1/2
SU-RMA	$(3/4)^m$	1/2	1/2
UU-RMA	$(3/4)^m$	$(7/8)^{m+\log m}$	$\left(\frac{(w+1)(4w-1)}{6w^2}\right)^{\frac{m+\log m}{\log w}}$
FB-RMA	$(1/2)^m$	$(3/4)^{m+\log m}$	$\left(1 - \left(\frac{w-1}{w}\right)^2\right)^{\frac{m+\log m}{\log w}}$

Table 7. Attacks success probability.